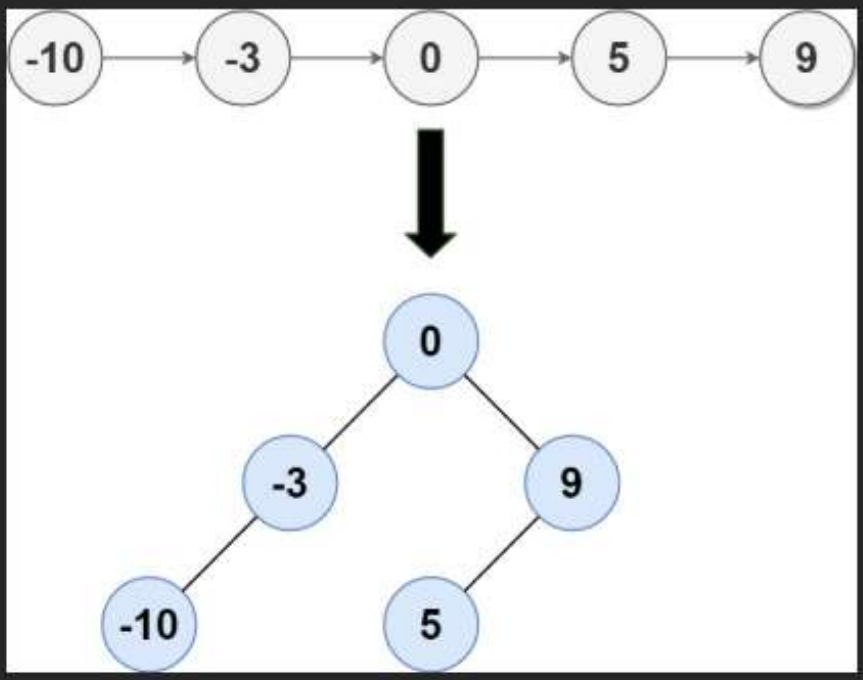
109. Convert Sorted List to Binary Search Tree

Medium | Topics | Companies

Given the `head` of a singly linked list where elements are sorted in **ascending order**, convert *it to a height-balanced binary search tree.*

Example 1:




```
Input: head = [-10,-3,0,5,9]
Output: [0,-3,9,-10,null,5]
Explanation: One possible answer is [0,-3,9,-10,null,5], which represents the shown height balanced BST.
```

Example 2:

```
Input: head = []
Output: []
```

Constraints:

- The number of nodes in `head` is in the range `[0, 2 * 10^4]`.
- `-10^5 <= Node.val <= 10^5`

Accepted 32 / 32 testcases passed

Akankshyaa submitted at Nov 02, 2026 16:37

Analysis Solution



Big Diwali Dhamaka

Unlock the Full Experience with Premium for ₹583.25/mo — Ends Nov 8!

Runtime

6 ms | Beats 47.99%

Memory

33.02 MB | Beats 52.20%



Code | C++

```cpp
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
```

```cpp
class Solution {
public:
    ListNode* curr;
    TreeNode* build(int n) {
        if(n<=0){
            return NULL;
        }
        TreeNode* leftSubtree= build(n/2);
        TreeNode* root= new TreeNode(curr->val);
        curr=curr->next;
        root->right= build(n-n/2-1);
        root->left=leftSubtree;
        return root;
    }
    TreeNode* sortedListToBST(ListNode* head){
        curr= head;
        int n=0;
        ListNode* temp= head;
        while(temp!=NULL){
            n++;
            temp=temp->next;
        }
        return build(n);
    }
};
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

Q:- Day 86 / 2-11-26

leetcode-109, convert sorted list to BST.

__/__/____

we're given a singly ll whose elements are already sorted in ascending order.
our task is to convert that sorted list into a height-balanced BST.

what is a sorted ll? A ll is a collection of nodes where each node stores. → a value
→ a pointer to the next node.

for ex
-10 → -3 → 0 → 5 → 9. → sorted, ascending order means every next element is greater than or equal to the previous element.

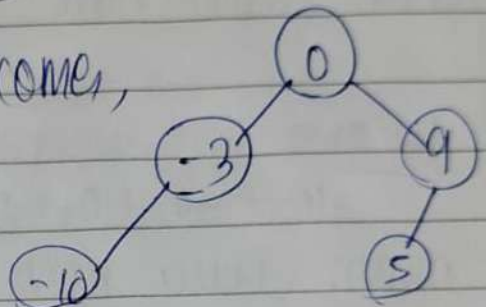
what is a height-balanced BST? A tree is height-balanced if, at every node, the difference between the heights of its left & right subtree is at most 1.
in simple, we don't want one side of the tree to become very long while the other side is very short.
our goal is to pick suitable middle elements as roots so that the resulting BST remains balanced.

example-1 input, head = [-10, -3, 0, 5, 9].
we have 5 sorted elements.
the middle element is 0.

So, we make 0 as the root.
- elements smaller than 0 go to the left.
- " greater " " " " " right.

Now left elements are -10, -3
Now right " " 5, 9.
We repeat the same process.
middle of [-10, -3] → -3
middle of [5, 9] → 9.

the final tree become,



O/P [0, -3, 9, -10, null, 5]
↳ the output is a level-order representation of the tree, not a new linked list.

Ex-2 input: head = []
output: [].

So, for the optimal approach, can we create a balanced BST directly from the ll without converting it into an array?
↳ yes, that's exactly what we'll do.

Key obs. ⇒ A BST's inorder traversal produces values in sorted ascending order.
leftsubtree → root → rightsubtree.

so, instead of moving randomly through the linked list to find middle elements, we can construct the BST in order sequence. —/—/—

class Solution { public:
    ListNode* curr; we declare a linked list pointer. It tells us which linked list node we are currently processing.

helper function ← TreeNode* build(int n) {
    G means the func^n will return a pointer to tree node.
    if(n<=0) { return NULL; } → base cond^n.
if there are no nodes left to construct, return NULL.
    TreeNode* leftSubtree = build(n/2);
we recursively construct the leftsubtree 1st.
why n/2? but approx. half of the nodes should go into the left subtree to keep the tree balanced.
    TreeNode* root = new TreeNode(curr->val);
after constructing the left subtree, the current linked list node becomes the root.
    curr = curr->next;
ListNode → nodes belonging to the original ll.
TreeNode → " " " " new binary tree we are creating.

initially, the curr pointer points to -10. then after constructing the left subtree, we have already moved curr forward twice. So, now curr currently points to 0.

cur->val means, give me the value stored in the node where cur is currently pointing. here, cur->val=c.

and now, new TreeNode(c); means, create a new tree node containing c.

* we 1st build the left subtree, then take the current le value to create the root & finally build the right subtree.

Root->right = build(n - n/2 - 1); we construct the right subtree, using this formula:

total nodes: n

left subtree nodes = n/2

Root node: 1.

therefore, right nodes = n - n/2 - 1.

Root->left = leftSubtree; we attach the already constructed left subtree to the Root

return Root; & finally, return the Root of the completed subtree.

mainfunction ← TreeNode* sortedListToBST(ListNode* head){ → receive the starting address of our original LL.

the func returns a pointer to the Root of our new BST.

cur = head; → To traverse the LL. declared cur, now initializing it.

int n = 0; → stores the no. of nodes in LL.

ListNode* temp = head;

I started at the 1st node, but we need to count the LL nodes without moving our actual cur pointer.

cur: will be used later to construct the BST.

temp: " " " used to count the nodes.

```
while (temp != NULL) {
    n++;
    temp = temp->next;
}
return build(n);
};
```

this loop visits every node in order.
every time the visit occurs, we ↑ the count by 1.
this means moving the temp pointer to the next node.
when temp reaches NULL, there are no more nodes, and the loop stops.

now comes the actual tree construction, we counted all the nodes.
the build() fcn knows where to start reading values from the ll. finally, return gives the constructed tree's ~~node~~ root back to sortedListToBST().

T.C = O(n) → visiting ll nodes takes O(n) + every node is visited exactly once during construction

auxiliary space = O(log n)

↓

the recursion stack stores calls corresponding to the height of the balanced tree.
since the height of a balanced BST is O(log n), space is O(log n).

↓

creating each tree node takes O(1).
total = O(n).

Complete binary tree

Input: -10 → -3 → 0 → 5 → 9.

__/__/__

S1 → total nodes = 5.

we call, build(5/2)

build(2)

we construct 2 nodes:

1st node: -10

2nd node: -3.

our left subtree: -3

-10

our current pointer, now points to 0.

S2 → now node = 0, current points to 5.

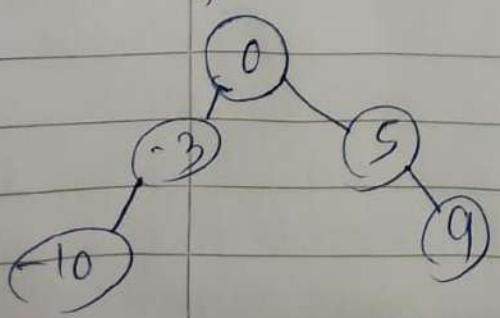
S3 → n - n/2 - 1

= 5 - 2 - 1

= 2

build(2),

this constructs the remaining 2 nodes - 5, 9,



→ the function does 3 things

- sets up our linked list pointer.
- counts the total number of nodes.
- calls build() to construct the bst.

sortedListToBST(head)

(receives the linked list)

↓

current = head

(set the construction pointer to the beginning).

↓

(count nodes using temp

n = 5

↓

returns build(5)

(constructs the BST using recursion).

↓

returns the root of the BST).